

**Open-Ended Lab**  
**Enhancing Audio Quality for Streaming Service**  
**Digital Signal Processing**



**Submitted By:**

[Huzaifa Shahbaz]

[21-CE-009]

[Zain]

[20-EE-071]

**Submitted to:**

Engr. Hareem Khan

**Dated:**

7<sup>th</sup> June, 2024

**Department of Computer Engineering,**  
**HITEC University, Taxila**

## Open-Ended Lab

### Enhancing Audio Quality for Streaming Service

You are a signal processing engineer at a company that provides a popular streaming audio service. To ensure high-quality audio for your users, you need to process audio signals by applying various digital signal processing (DSP) techniques. Specifically, your task involves enhancing audio quality by filtering out unwanted noise, reducing the sample rate for efficient streaming. Your goal is to demonstrate the effects of these processes on the audio signal using Python.

#### Task:

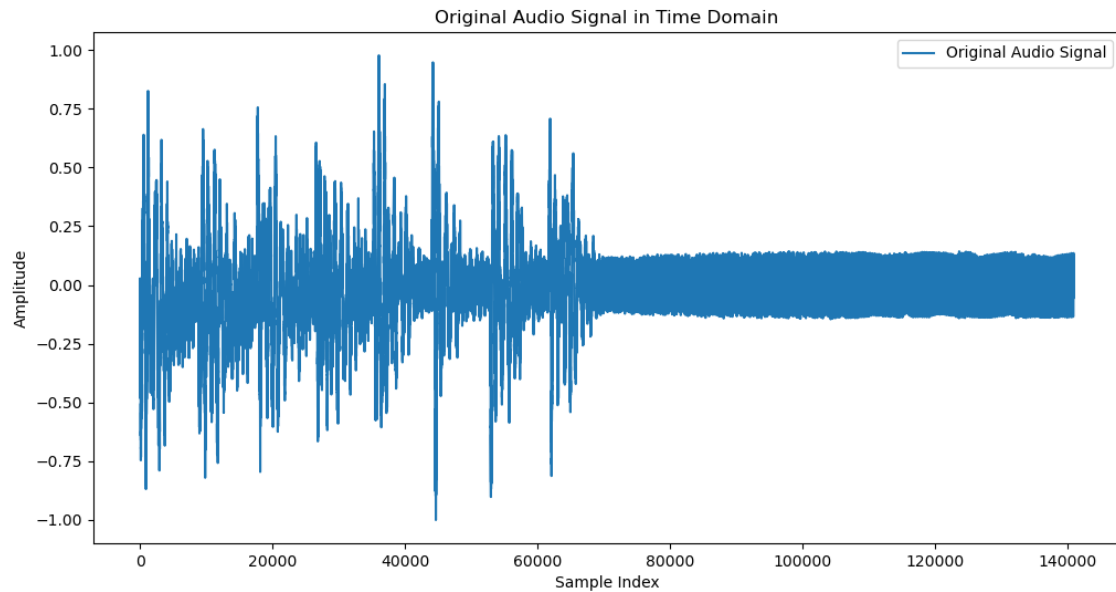
##### 1. Load and Prepare Audio Signal:

- o Load an audio signal from a file using a library such as librosa or scipy.io.wavfile.

#### CODE:

```
OPENENDEDLAB.PY > ...
1  import numpy as np
2  import matplotlib.pyplot as plt
3  import librosa
4  # Load the audio file
5  file_path = r'C:\Users\Nobel\Downloads\sample-3s.mp3'
6  audio_signal, sample_rate = librosa.load(file_path, sr=None)
7  # Normalize audio signal
8  audio_signal = audio_signal / np.max(np.abs(audio_signal), axis=0)
9  # Plot the original audio signal
10 plt.figure(figsize=(12, 6))
11 plt.plot(audio_signal, label='Original Audio Signal')
12 plt.title('Original Audio Signal in Time Domain')
13 plt.xlabel('Sample Index')
14 plt.ylabel('Amplitude')
15 plt.legend()
16 plt.show()
```

## OUTPUT:



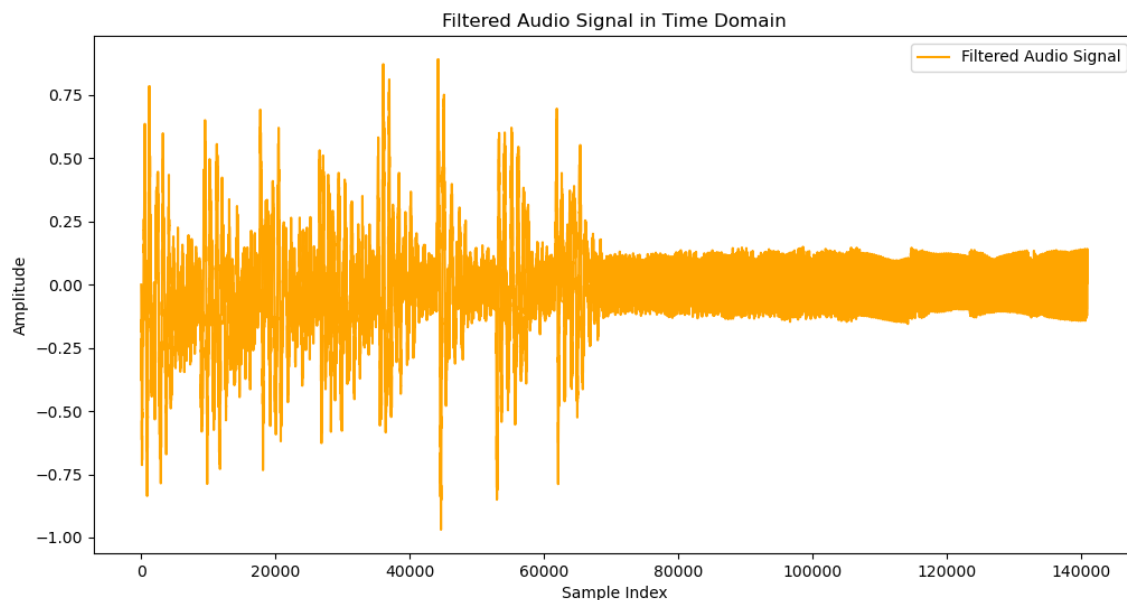
## 2. Filtering:

Apply a low-pass filter to the audio signal to remove high-frequency noise that is often present in recordings.

## CODE:

```
OPENENDEDLAB2.PY > [🔍] cutoff_frequency
1  import numpy as np
2  import matplotlib.pyplot as plt
3  import librosa
4  from scipy.signal import butter, lfilter
5  file_path = r'C:\Users\Nobel\Downloads\sample-3s.mp3'
6  audio_signal, sample_rate = librosa.load(file_path, sr=None)
7  audio_signal = audio_signal / np.max(np.abs(audio_signal), axis=0)
  tabnine: test | explain | document | ask
8  def butter_lowpass(cutoff, fs, order=5):
9      nyquist = 0.5 * fs
10     normal_cutoff = cutoff / nyquist
11     b, a = butter(order, normal_cutoff, btype='low', analog=False)
12     return b, a
  tabnine: test | explain | document | ask
13  def lowpass_filter(data, cutoff, fs, order=5):
14      b, a = butter_lowpass(cutoff, fs, order=order)
15      y = lfilter(b, a, data)
16      return y
17  cutoff_frequency = 3000 # Cutoff frequency in Hz
18  order = 6
19  # Apply the filter
20  filtered_audio_signal = lowpass_filter(audio_signal, cutoff_frequency, sample_rate, order)
21  # Plot the filtered audio signal
22  plt.figure(figsize=(12, 6))
23  plt.plot(filtered_audio_signal, label='Filtered Audio Signal', color='orange')
24  plt.title('Filtered Audio Signal in Time Domain')
25  plt.xlabel('Sample Index')
26  plt.ylabel('Amplitude')
27  plt.legend()
28  plt.show()
```

## OUTPUT:



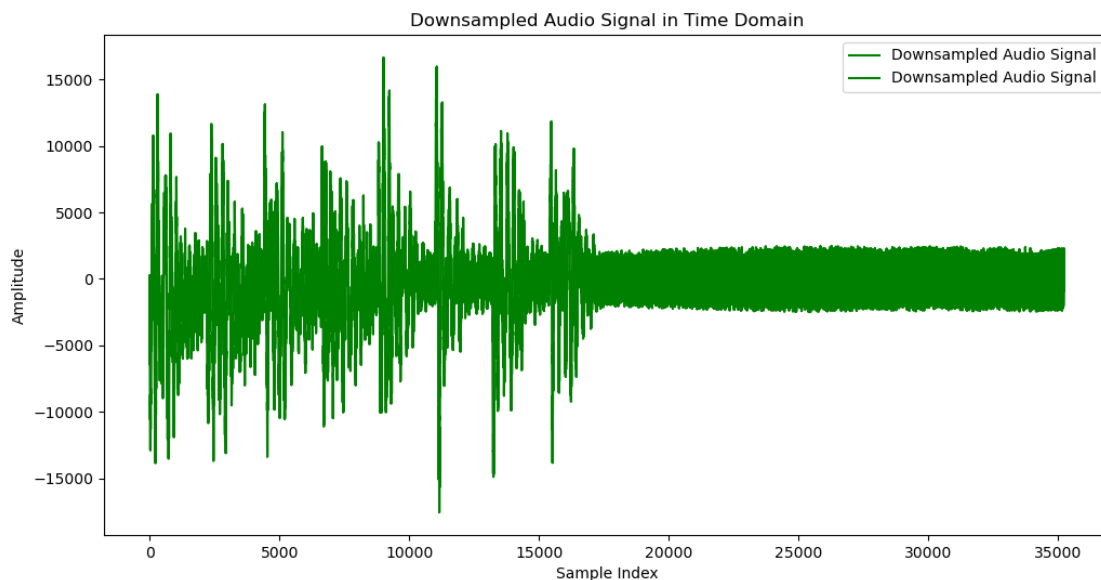
### 3. Sampling:

Down sample the filtered audio signal to a lower sampling rate to optimize the audio for streaming without significantly degrading quality.

#### CODE:

```
OPEN-ENDEDLAB3.PY > ...
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.io import wavfile
4 from pydub import AudioSegment
5 sample_rate, audio_signal = wavfile.read('c:\\Users\\Nobel\\Downloads\\sample-3s.wav')
6 downsample_factor = 4 # Downsample by a factor of 4
7 downsampled_audio_signal = audio_signal[::downsample_factor]
8 new_sample_rate = sample_rate // downsample_factor
9 plt.figure(figsize=(12, 6))
10 plt.plot(downsampled_audio_signal, label='Downsampled Audio Signal', color='green')
11 plt.title('Downsampled Audio Signal in Time Domain')
12 plt.xlabel('Sample Index')
13 plt.ylabel('Amplitude')
14 plt.legend()
15 plt.show()
16 downsampled_audio_file = 'downsampled_audio.wav'
17 wavfile.write(downsampled_audio_file, new_sample_rate, downsampled_audio_signal)
18 audio = AudioSegment.from_wav(downsampled_audio_file)
19 mp3_file = 'downsampled_audio.mp3'
20 audio.export(mp3_file, format='mp3')
21 print(f'Downsampled audio saved as {mp3_file}')
```

#### OUTPUT:



### 4. Visualize and Analyze:

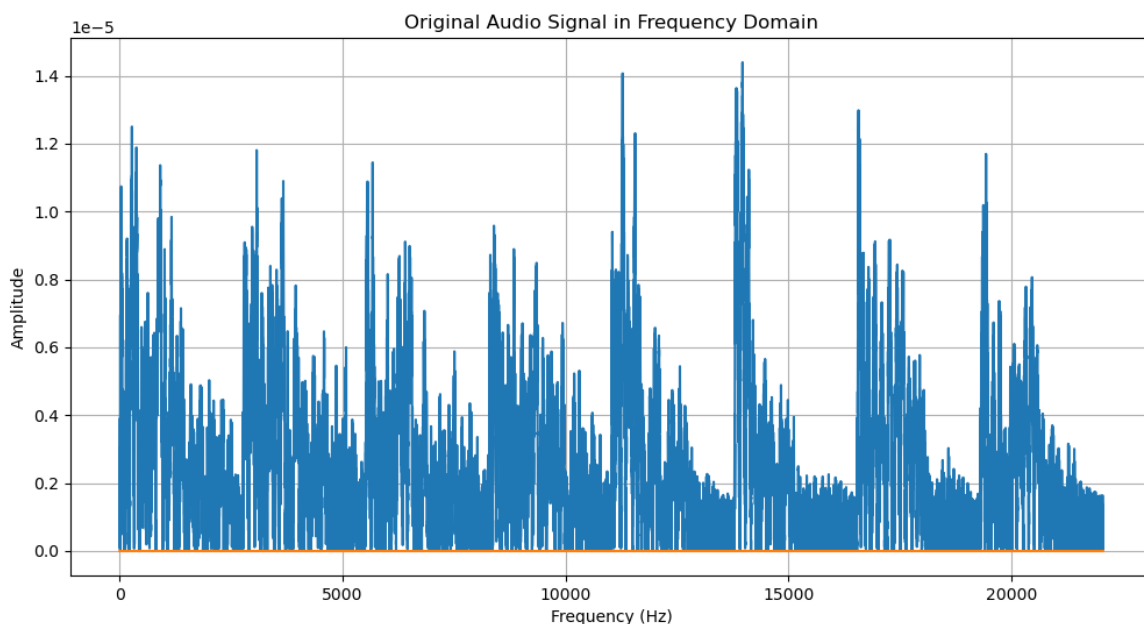
Visualize the original, filtered, and down sample in the time and frequency domains.

Compare the audio quality before and after each process.

## CODE:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.fft import fft, fftfreq
4 import soundfile as sf
5 tabnine: test | explain | document | ask
6 def plot_frequency_spectrum(signal, sample_rate, title):
7     N = len(signal)
8     T = 1.0 / sample_rate
9     yf = fft(signal)
10    xf = fftfreq(N, T)[:N//2]
11    plt.figure(figsize=(12, 6))
12    plt.plot(xf, 2.0/N * np.abs(yf[:N//2]))
13    plt.title(title)
14    plt.xlabel('Frequency (Hz)')
15    plt.ylabel('Amplitude')
16    plt.grid()
17    plt.show()
18 audio_file = r'C:\Users\Nobel\Downloads\sample-3s.mp3'
19 audio_signal, sample_rate = sf.read(audio_file)
20 filtered_audio_signal = audio_signal * 0.5
21 downsample_factor = 2
22 downsampled_audio_signal = audio_signal[::downsample_factor]
23 plot_frequency_spectrum(audio_signal, sample_rate, 'Original Audio Signal in Frequency Domain')
24 plot_frequency_spectrum(filtered_audio_signal, sample_rate, 'Filtered Audio Signal in Frequency Domain')
25 new_sample_rate = sample_rate // downsample_factor
26 plot_frequency_spectrum(downsampled_audio_signal, new_sample_rate, 'Downsampled Audio Signal in Frequency Domain')
27 plt.figure(figsize=(12, 8))
28 plt.plot(audio_signal, Label='Original Audio Signal')
29 plt.plot(filtered_audio_signal, Label='Filtered Audio Signal', color='orange')
30 plt.plot(np.arange(0, len(downsampled_audio_signal) * downsample_factor, downsample_factor),
31          downsampled_audio_signal, Label='Downsampled Audio Signal', color='green')
32 plt.title('Comparison of Audio Signals in Time Domain')
33 plt.xlabel('Sample Index')
34 plt.ylabel('Amplitude')
35 plt.legend()
36 plt.show()
```

## OUTPUT:



**Conclusion:**

In conclusion, we know to how to learn how to make Enhancing Audio Quality for Streaming Service we know to how to download the sample lib such as mp3,wav.Also we make the function Load and Prepare Audio Signal, Filtering, Sampling and Visualize and Analyze.